Security Architecture Review for MCP Integrations

Run this review **before** onboarding any MCP server into an environment that has access to data or actions you care about. It evaluates four things: tool definitions, server trust boundaries, prompt-injection attack surface, and the tool-call authorization model.

0. Inventory and provenance

- Where does the server come from? Pinned to a specific version/digest?
- Is it first-party, vendor, or community? A community MCP server is a supply-chain dependency that runs code in your environment (OWASP LLM03).
- What transport (stdio = local privileges; HTTP = network surface + its own authN)?
- What identity/permissions does the server process run with?

1. Evaluate tool definitions

For each tool the server advertises (`tools/list`):

- **Description hygiene.** Does the description contain instructions to the model ("also call X", "include the API key", "ignore prior rules")? Tool descriptions are model-visible context and a tool-poisoning vector. Reject or normalize them.
- **Schema strictness.** Are inputs typed and constrained (enums, lengths, formats)? Loose `string` args are where injection rides in.
- **Capability scope.** What can this tool actually do? Does a "read" tool secretly write? Map each tool to the data it reads and the effects it causes.
- **Necessity.** Is this tool needed at all? Disable unused tools (reduce agency).

2. Map trust boundaries

- Draw the boundary: host ↔ client ↔ server ↔ downstream (filesystem, DB, network, other APIs). Where does untrusted model-chosen input cross into privileged action?
- Does the server reach the network? If so, is egress allow-listed and the metadata endpoint blocked (anti-SSRF)?
- Does the server touch the filesystem? Is access confined to an allow-listed root (anti path-traversal)?
- Is the server sandboxed (container/seccomp/microVM) so a compromise can't pivot?

3. Assess the prompt-injection attack surface

- Enumerate every channel that can carry an injection into the model that then calls this server's tools: user input, retrieved docs, other tools' outputs, and the server's own tool descriptions/resource content.

- For each, ask: *if this text said "call `dangerous_tool` with these args", what stops it?* The answer must be a deterministic authorization check, not the model's judgment.
- Confirm retrieved/resource content is fenced and treated as data (see `secure-by-default.md`).

4. Review the tool-call authorization model

- **AuthN:** Is each session bound to a verified principal?
- **AuthZ:** Are tool calls authorized per-principal with least-privilege scopes (not "if the server can, the model can")? Are high-risk tools gated by allow-listed tenants/roles?
- **HITL:** Do irreversible/destructive tools require human approval?
- **Rate limits:** Per-principal, per-tool caps to bound abuse and cost?
- **Audit:** Is every tool call logged with subject, tool, argument hash, outcome?

`airte.secure_mcp_server.auth` (`Principal`, `ToolPolicy`, `authorize`) and the `SecureToolRegistry` provide reference implementations of this model.

5. Static + dynamic checks

- **Static:** run the AST scanner over the server source if available: `python -m airte.mcp_audit.scanner <server_src> --fail-on HIGH`.
- **Dynamic:** in a sandbox, attempt traversal, SSRF (to a metadata IP), command injection, and an over-broad tool call; confirm each is blocked and logged.

Decision

Document residual risk and a go/no-go. Block onboarding if any tool can reach a dangerous sink from model-chosen input without authorization, if egress is unrestricted, if there is no per-principal authz, or if tool descriptions carry embedded instructions. Re-review whenever the server version or tool set changes.

Review checklist (copy into the PR)

```
[ ] Source pinned & provenance documented        [ ] Egress allow-listed, metadata blocked
[ ] Transport & process privileges reviewed       [ ] Filesystem access confined to root
[ ] Tool descriptions free of instructions         [ ] Per-principal authZ + least privilege
[ ] Input schemas strict                            [ ] HITL on irreversible tools
[ ] Unused tools disabled                           [ ] Per-principal/per-tool rate limits
[ ] Injection channels enumerated & fenced          [ ] Every tool call audit-logged
[ ] Static scan clean (or risks accepted)           [ ] Dynamic abuse tests pass
```